MLA0303_RL_PROGRAM_14.py

```
"""
Experiment 14: Actor-Critic (A2C) for a smart elevator scheduling
system to reduce passenger waiting time. A3C extends A2C by running
several such actor-critic workers in parallel on copies of the
environment and periodically averaging their gradients into a shared
model; the single-worker update rule implemented below is identical
to what each A3C worker performs locally.

State  = (elevator_floor, call_floor)   Floors: 0-4
Actions = UP, DOWN, STOP(open doors)
"""

import numpy as np

np.random.seed(2)

N_FLOORS = 5
ACTIONS = ["UP", "DOWN", "STOP"]
GAMMA = 0.95
LR_ACTOR = 0.05
LR_CRITIC = 0.1
EPISODES = 2500


class ElevatorEnv:
    def reset(self):
        self.elevator = 0
        self.call = np.random.randint(1, N_FLOORS)
        return self._state()

    def _state(self):
        return self.elevator * N_FLOORS + self.call

    def step(self, action):
        reward, done = -1, False
        if action == 0:
            self.elevator = min(N_FLOORS - 1, self.elevator + 1)
        elif action == 1:
            self.elevator = max(0, self.elevator - 1)
        else:                       # STOP
            if self.elevator == self.call:
                reward, done = 15, True
            else:
                reward = -5
        return self._state(), reward, done


N_STATES = N_FLOORS * N_FLOORS
theta = np.zeros((N_STATES, len(ACTIONS)))     # actor (policy) parameters
V = np.zeros(N_STATES)                          # critic (state-value) parameters


def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()


def train():
    env = ElevatorEnv()
    reward_history = []
    for ep in range(EPISODES):
        state = env.reset()
        total = 0
        for _ in range(30):
            probs = softmax(theta[state])
            action = np.random.choice(len(ACTIONS), p=probs)
            next_state, reward, done = env.step(action)

            td_target = reward + (0 if done else GAMMA * V[next_state])
            td_error = td_target - V[state]              # advantage estimate

            V[state] += LR_CRITIC * td_error             # critic update
            grad = -probs
            grad[action] += 1
            theta[state] += LR_ACTOR * td_error * grad     # actor update

            state = next_state
            total += reward
            if done:
                break
        reward_history.append(total)
    return env, reward_history


if __name__ == "__main__":
    env, rewards = train()
    print(f"Average return (first 50 episodes): {np.mean(rewards[:50]):.2f}")
    print(f"Average return (last 50 episodes) : {np.mean(rewards[-50:]):.2f}")

    for call_floor in [3, 4]:
        env.elevator, env.call = 0, call_floor
        state = env._state()
        path = [env.elevator]
        for _ in range(15):
            action = int(np.argmax(theta[state]))
            state, _, done = env.step(action)
            path.append(env.elevator)
            if done:
                break
```

```
print(f"Call from floor {call_floor}: elevator path {path}")
```

PROGRAM OUTPUT

```
Average return (first 50 episodes): -0.60
Average return (last 50 episodes) : 12.48
Call from floor 3: elevator path [0, 1, 2, 3, 3]
Call from floor 4: elevator path [0, 1, 2, 3, 4, 4]
```